DSA Streak: Daily LeetCode in Java

2026-11-12 | 5 problems

#	Problem	Difficulty	Topic
1	Coin Change	Medium	Array, Dynamic Programming
2	Linked List Cycle	Easy	Hash Table, Linked List
3	Kth Largest Element in an Array	Medium	Array, Divide and Conquer
4	Subarray Sum Equals K	Medium	Array, Hash Table
5	Word Ladder	Hard	Hash Table, String

1. Coin Change

Medium | #322 | Array, Dynamic Programming, Breadth-First Search
https://leetcode.com/problems/coin-change/

Problem

You get coin denominations (unlimited supply of each) and a target amount. Return the fewest coins that add up to exactly the amount, or -1 if it cannot be made.

Examples

Example 1	
Input	`coins = [1,2,5], amount = 11`
Output	`3`
Why	11 = 5 + 5 + 1

Example 2	
Input	`coins = [2], amount = 3`
Output	`-1`
Why	Only 2s are available, so an odd amount is impossible.

Example 3	
Input	`coins = [1], amount = 0`
Output	`0`
Why	Zero coins make amount 0.

Intuition

Greedy (always take the biggest coin) fails for coins like [1,3,4] and amount 6. Instead, the best answer for amount i comes from trying every coin c as the last coin: 1 + best answer for i - c. Small amounts are solved first, so each dp[i] only looks at already solved values.

Approach

1. Make dp of size amount + 1 and fill it with amount + 1, which acts as infinity (no answer can use more than amount coins).
2. Set dp[0] = 0.
3. For i from 1 to amount, for every coin c <= i, set dp[i] = min(dp[i], dp[i - c] + 1).
4. If dp[amount] is still bigger than amount, return -1, else return dp[amount].

Dry run on Example 1

- coins = [1,2,5], amount = 11, dp filled with 12, dp[0] = 0
- i=1: c=1 gives dp[0]+1 = 1, so dp[1] = 1
- i=2: c=1 gives 2, c=2 gives dp[0]+1 = 1, so dp[2] = 1
- i=3: dp[2]+1 = 2 and dp[1]+1 = 2, so dp[3] = 2
- i=4: dp[2]+1 = 2, so dp[4] = 2
- i=5: c=5 gives dp[0]+1 = 1, so dp[5] = 1
- i=6: dp[5]+1 = 2, dp[6] = 2. i=7: dp[5]+1 = 2, dp[7] = 2. i=8: dp[3]+1 = 3, dp[8] = 3
- i=9: dp[7]+1 = 3, dp[9] = 3. i=10: dp[5]+1 = 2, dp[10] = 2
- i=11: c=1 gives dp[10]+1 = 3, c=2 gives dp[9]+1 = 4, c=5 gives dp[6]+1 = 3, so dp[11] = 3
- dp[11] = 3 is not more than 11, return 3

Java solution

```
import java.util.*;

class Solution {
    public int coinChange(int[] coins, int amount) {
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, amount + 1);
        dp[0] = 0;
        for (int i = 1; i <= amount; i++) {
            for (int c : coins) {
                if (c <= i) dp[i] = Math.min(dp[i], dp[i - c] + 1);
            }
        }
        return dp[amount] > amount ? -1 : dp[amount];
    }
}
```

Complexity

Time: O(amount * number of coins) Space: O(amount)

Edge cases handled

- amount = 0 returns 0 directly from dp[0]
- Impossible amount (coins = [2], amount = 3) stays at amount + 1 and returns -1
- Huge coin like 2147483647 is skipped by the c <= i check, so no overflow
- Single coin 1 with amount 10000 gives 10000

Pattern to remember

Unbounded knapsack: dp[i] = min over choices of dp[i - choice] + 1, loop amounts from small to large.

2. Linked List Cycle

Easy | #141 | Hash Table, Linked List, Two Pointers
https://leetcode.com/problems/linked-list-cycle/

Problem

Given the head of a linked list, return true if following next pointers ever brings you back to a node you already visited (a cycle), otherwise false. pos is only used by the judge to build the input.

Examples

Example 1	
Input	`head = [3,2,0,-4], pos = 1`
Output	`true`
Why	The tail connects back to the node at index 1.

Example 2	
Input	`head = [1,2], pos = 0`
Output	`true`
Why	The tail connects back to the first node.

Example 3	
Input	`head = [1], pos = -1`
Output	`false`
Why	There is no cycle.

Intuition

Floyd's tortoise and hare. A slow pointer moves 1 step and a fast pointer moves 2 steps. Without a cycle, fast simply runs off the end. With a cycle, both end up looping inside it and fast gains 1 node per step on slow, so it must land exactly on slow.

Approach

1. Start slow and fast at head.
2. While fast and fast.next are not null, move slow one step and fast two steps.
3. If slow == fast at any point, return true.
4. If the loop ends, fast hit null, so return false.

Dry run on Example 1

- List 3 -> 2 -> 0 -> -4 and -4 points back to 2. slow = 3, fast = 3
- Step 1: slow = 2, fast = 0. Not equal
- Step 2: slow = 0, fast moves -4 then 2, so fast = 2. Not equal
- Step 3: slow = -4, fast moves 0 then -4, so fast = -4. slow == fast, return true

Java solution

```
class Solution {
    public boolean hasCycle(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) return true;
        }
        return false;
```

```
    }
}
```

Complexity

Time: O(n) Space: O(1)

Edge cases handled

- Empty list: fast is null, return false
- Single node with no cycle returns false
- Single node pointing to itself returns true
- Duplicate values do not matter because we compare node references, not values

Pattern to remember

Fast and slow pointers detect cycles in O(1) space; the same trick finds the middle of a list.

3. Kth Largest Element in an Array

Medium | #215 | Array, Divide and Conquer, Sorting, Heap (Priority Queue), Quickselect
https://leetcode.com/problems/kth-largest-element-in-an-array/

Problem

Return the kth largest element of the array in sorted order (duplicates count separately, so it is not the kth distinct value). The follow-up asks to do it without fully sorting.

Examples

Example 1	
Input	`nums = [3,2,1,5,6,4], k = 2`
Output	`5`
Why	Sorted descending: 6,5,4,3,2,1. The 2nd one is 5.

Example 2	
Input	`nums = [3,2,3,1,2,4,5,5,6], k = 4`
Output	`4`
Why	Sorted descending: 6,5,5,4,... The 4th one is 4.

Intuition

Keep only the k largest numbers seen so far in a min-heap. The smallest of those k sits on top, and that is exactly the kth largest. Any number smaller than the top gets pushed and immediately popped.

Approach

1. Create a min-heap pq.
2. For each x in nums, offer x into pq.
3. If pq.size() > k, poll the smallest.
4. After the loop, pq holds the k largest values, return pq.peek().

Dry run on Example 1

- nums = [3,2,1,5,6,4], k = 2
- x=3: pq = {3}. x=2: pq = {2,3}
- x=1: pq = {1,2,3}, size 3 > 2, poll 1, pq = {2,3}
- x=5: pq = {2,3,5}, poll 2, pq = {3,5}
- x=6: pq = {3,5,6}, poll 3, pq = {5,6}
- x=4: pq = {4,5,6}, poll 4, pq = {5,6}
- return pq.peek() = 5

Java solution

```
import java.util.*;

class Solution {
    public int findKthLargest(int[] nums, int k) {
        PriorityQueue<Integer> pq = new PriorityQueue<>();
        for (int x : nums) {
            pq.offer(x);
            if (pq.size() > k) pq.poll();
        }
        return pq.peek();
    }
}
```

Complexity

Time: O(n log k) Space: O(k)

Edge cases handled

- Single element with k = 1
- All duplicates like [2,2,2,2]
- Negative numbers
- k = n returns the minimum, k = 1 returns the maximum

Pattern to remember

Top k largest means a min-heap of size k (top k smallest means a max-heap). Quickselect gives O(n) average if asked.

4. Subarray Sum Equals K

Medium | #560 | Array, Hash Table, Prefix Sum
https://leetcode.com/problems/subarray-sum-equals-k/

Problem

Count how many contiguous, non-empty subarrays have a sum exactly equal to k. Numbers can be negative.

Examples

Example 1	
Input	`nums = [1,1,1], k = 2`
Output	`2`
Why	[1,1] starting at index 0 and [1,1] starting at index 1.

Example 2	
Input	`nums = [1,2,3], k = 3`
Output	`2`
Why	[1,2] and [3].

Intuition

Let sum be the prefix sum up to the current index. A subarray ending here sums to k exactly when some earlier prefix equals sum - k. So keep a map from prefix sum to how many times it has appeared and add that count at each step. Sliding window does not work because of negative numbers.

Approach

1. map stores prefix sum to frequency. Put (0, 1) for the empty prefix.
2. Walk through nums keeping a running sum.
3. Add map.getOrDefault(sum - k, 0) to ans.
4. Then increase the count of sum in map.
5. Return ans.

Dry run on Example 1

- nums = [1,1,1], k = 2. map = {0:1}, sum = 0, ans = 0
- x=1: sum = 1, sum - k = -1 not in map, ans = 0, map = {0:1, 1:1}
- x=1: sum = 2, sum - k = 0 found once, ans = 1, map = {0:1, 1:1, 2:1}
- x=1: sum = 3, sum - k = 1 found once, ans = 2, map adds 3:1
- return 2

Java solution

```
import java.util.*;

class Solution {
    public int subarraySum(int[] nums, int k) {
        Map<Integer, Integer> map = new HashMap<>();
        map.put(0, 1);
        int sum = 0, ans = 0;
        for (int x : nums) {
            sum += x;
            ans += map.getOrDefault(sum - k, 0);
            map.put(sum, map.getOrDefault(sum, 0) + 1);
        }
        return ans;
    }
}
```

Complexity

Time: O(n) Space: O(n)

Edge cases handled

- Single element equal to k counts 1
- All zeros with k = 0: [0,0,0] gives 6
- Negative numbers like [1,-1,0] with k = 0 give 3
- The (0,1) seed is what counts subarrays that start at index 0

Pattern to remember

Count subarrays with a target sum: prefix sum + HashMap of seen prefix counts, look up sum - k before inserting sum.

5. Word Ladder

Hard | #127 | Hash Table, String, Breadth-First Search
https://leetcode.com/problems/word-ladder/

Problem

Change beginWord into endWord one letter at a time, where every intermediate word must be in wordList. Return the number of words in the shortest such sequence (including both ends), or 0 if it is impossible. beginWord does not need to be in the list.

Examples

Example 1	
Input	`beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log","cog"]`
Output	`5`
Why	hit -> hot -> dot -> dog -> cog has 5 words.

Example 2	
Input	`beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log"]`
Output	`0`
Why	endWord cog is not in wordList, so no valid sequence exists.

Intuition

Every word is a node and two words are connected when they differ by one letter. Shortest path in an unweighted graph means BFS, level by level. Instead of comparing every pair of words, generate neighbours by trying all 26 letters at each position and checking a HashSet.

Approach

1. Put wordList in set. If endWord is not in set, return 0.
2. Start q with beginWord, steps = 1.
3. Process the queue one level at a time (sz = q.size()).
4. For each word cur, try every position i and every letter c, build w.
5. If w is in set: return steps + 1 if it is endWord, otherwise remove it from set (marks visited) and add to q.
6. After each level, steps++. If the queue empties, return 0.

Dry run on Example 1

- set has all 6 words, q = [hit], steps = 1
- Level 1: hit, changing i=1 to o gives hot. Add hot, remove from set. q = [hot], steps = 2
- Level 2: hot, i=0 gives dot and lot. q = [dot, lot], steps = 3
- Level 3: dot, i=2 gives dog. lot, i=2 gives log. q = [dog, log], steps = 4
- Level 4: dog, i=0 with c = 'c' gives cog, which is endWord, return steps + 1 = 5

Java solution

```
import java.util.*;

class Solution {
    public int ladderLength(String beginWord, String endWord, List<String> wordList) {
        Set<String> set = new HashSet<>(wordList);
        if (!set.contains(endWord)) return 0;
        Queue<String> q = new LinkedList<>();
        q.add(beginWord);
        set.remove(beginWord);
        int steps = 1;
```

```
        while (!q.isEmpty()) {
            int sz = q.size();
            for (int s = 0; s < sz; s++) {
                char[] cur = q.poll().toCharArray();
                for (int i = 0; i < cur.length; i++) {
                    char old = cur[i];
                    for (char c = 'a'; c <= 'z'; c++) {
                        if (c == old) continue;
                        cur[i] = c;
                        String w = new String(cur);
                        if (set.contains(w)) {
                            if (w.equals(endWord)) return steps + 1;
                            set.remove(w);
                            q.add(w);
                        }
                    }
                    cur[i] = old;
                }
            }
            steps++;
        }
        return 0;
    }
}
```

Complexity

Time: O(N * L * 26) where N is the list size and L the word length Space: O(N * L)

Edge cases handled

- endWord missing from wordList returns 0
- beginWord and endWord one letter apart returns 2
- Duplicates in wordList are handled by the set
- beginWord inside wordList is removed so it is not revisited

Pattern to remember

Shortest transformation or fewest moves in an implicit graph: BFS level by level, generate neighbours on the fly, remove from the set to mark visited.